

Name: Gayatri Gupta

Internship: Future Interns

Task 1: Vulnerability

Assessment Report

EXECUTIVE SUMMARY

This Vulnerability Assessment was conducted on the web application `demo.testfire.net` to identify potential security vulnerabilities and evaluate the overall security posture of the application. The assessment was performed using industry-standard security tools including Nmap and OWASP ZAP.

The objective of this assessment was to discover security weaknesses that could potentially be exploited by malicious actors and to provide recommendations for improving the security of the application. The testing process included network reconnaissance, service identification, vulnerability scanning, and analysis of web application security controls.

The assessment identified several vulnerabilities of varying severity levels, including SQL Injection, Cross-Site Scripting (XSS), absence of Anti-CSRF protection, missing security headers, and information disclosure issues. These vulnerabilities may expose the application to unauthorized access, data compromise, and malicious attacks if left unaddressed.

Based on the findings, appropriate remediation measures have been recommended to strengthen the security posture of the application and reduce potential security risks.

SCOPE AND METHODOLOGY

Scope-

The scope of this assessment was limited to the publicly accessible web application demo.testfire.net. The assessment focused on identifying common web application vulnerabilities, exposed services, security misconfigurations, and potential weaknesses that could impact the confidentiality, integrity, and availability of the application.

Target Application:

- Website: <http://demo.testfire.net>
- Assessment Type: Vulnerability Assessment
- Testing Approach: Authorized Security Testing on a Vulnerable Training Environment

Methodology-

The assessment was conducted using the following methodology:

1. Information Gathering and Reconnaissance- Nmap was used to identify open ports, active services, and exposed technologies running on the target server.
2. Service Enumeration- Discovered services were analyzed to identify server information and potential security exposures.
3. Vulnerability Assessment- OWASP ZAP was utilized to scan the web application and identify security vulnerabilities including injection flaws, cross-site scripting vulnerabilities, missing security controls, and configuration weaknesses.
4. Risk Analysis- Each vulnerability was evaluated and categorized according to its potential impact and severity level (High, Medium, or Low).
5. Remediation Recommendations- Appropriate mitigation strategies and security recommendations were provided for each identified vulnerability to reduce security risks and improve application security.

NMAP SCAN RESULTS

Tool Used: Nmap Version 7.99

Command Executed: nmap -sV demo.testfire.net

Findings: Port 80/TCP

Status: Open

Service: HTTP

Version: Apache Tomcat/Coyote JSP Engine 1.1

Port 443/TCP

Status: Open

Service: HTTPS

Version: Apache Tomcat/Coyote JSP Engine 1.1

Port 8080/TCP

Status: Open

Service: HTTP Alternate

Version: Apache Tomcat/Coyote JSP Engine 1.1

Port 8443/TCP

Status: Closed

Observations:

- Multiple web service ports were exposed.
- Server technology information was publicly visible.
- Open services increase the application's attack surface.
- Service version disclosure may assist attackers in identifying known vulnerabilities.

Risk Level: Medium

VULNERABILITY FINDINGS

SQL Injection (High Risk)

Description: SQL Injection vulnerabilities were identified during the assessment. This vulnerability occurs when user input is not properly validated before being included in database queries.

Business Impact: An attacker may gain unauthorized access to sensitive data, modify records, bypass authentication mechanisms, or potentially compromise the database server.

Risk Level: High

Recommendation:

- Use parameterized queries and prepared statements.
- Validate and sanitize all user inputs.
- Implement least-privilege access for database accounts.
- Conduct regular security testing.

Cross-Site Scripting (XSS) (High Risk)

Description: Reflected Cross-Site Scripting vulnerabilities were detected. User-controlled input may be executed as script code within a victim's browser.

Business Impact: Attackers may steal session cookies, redirect users to malicious websites, or execute unauthorized actions on behalf of users.

Risk Level: High

Recommendation:

- Implement proper input validation.
- Apply output encoding before displaying user-generated content.
- Use Content Security Policy (CSP) headers.
- Perform regular code reviews.

Absence of Anti-CSRF Tokens (Medium Risk)

Description: Certain forms lacked Anti-CSRF protection mechanisms.

Business Impact: Attackers may trick authenticated users into performing unintended actions without their knowledge.

Risk Level: Medium

Recommendation:

- Implement Anti-CSRF tokens on all sensitive forms.
- Verify requests using server-side validation.
- Use SameSite cookie attributes where appropriate.

Missing Content Security Policy Header (Low Risk)

Description: The application does not define a Content Security Policy header.

Business Impact: The absence of CSP increases the risk of script injection attacks and makes exploitation of XSS vulnerabilities easier.

Risk Level: Low

Recommendation:

- Configure an appropriate Content Security Policy.
- Restrict script execution to trusted sources.

Missing Anti-Clickjacking Header (Low Risk)

Description: The application does not implement frame protection headers.

Business Impact: Attackers may embed the application within malicious websites and trick users into performing unintended actions.

Risk Level: Low

Recommendation:

- Implement X-Frame-Options or CSP frame-ancestors directives.

Information Disclosure and Server Information Exposure (Low Risk)

Description: Server information and technical details were exposed during assessment.

Business Impact: Exposed information may assist attackers during reconnaissance

Risk Summary Table

Vulnerability	Risk
SQL Injection	High
Cross-Site Scripting (XSS)	High
Missing Anti-CSRF Tokens	Medium
Missing CSP Header	Low
Missing Anti-Clickjacking Header	Low
Information Disclosure	Low
Server Information Exposure	Low

SECURITY RECOMMENDATIONS

Based on the findings of this assessment, the following actions are recommended:

1. Implement secure coding practices across all application modules.
2. Validate and sanitize all user-supplied input.
3. Use prepared statements for database interactions.
4. Implement Anti-CSRF protection on all sensitive forms.
5. Configure Content Security Policy (CSP) headers.
6. Enable Clickjacking protection through X-Frame-Options.
7. Remove unnecessary server and version disclosure information.
8. Conduct regular vulnerability assessments and penetration testing.
9. Maintain up-to-date software and security patches.
10. Provide security awareness training to development teams.

Implementing these recommendations will significantly improve the security posture of the application and reduce the likelihood of successful attacks.

CONCLUSION

A vulnerability assessment was successfully conducted on the target web application using Nmap and OWASP ZAP. The assessment identified multiple security weaknesses including SQL Injection, Cross-Site Scripting (XSS), missing Anti-CSRF protections, and several security configuration issues.

The findings demonstrate the importance of implementing secure coding practices, proper security controls, and continuous monitoring. High-risk vulnerabilities should be addressed immediately, while medium and low-risk findings should be incorporated into the organization's security improvement plan.

Regular security assessments, timely patch management, and adherence to security best practices are essential for maintaining a secure and resilient web application environment.